

# **Software Requirements Specification**

**for**

## **Event Management System**

**Version 1.0 approved**

**Prepared by**

**TEAM-7:**

Shayna Zaidi Gupta (PES2UG24CS465)

Shriya C Rao (PES2UG24CS496)

Sham Santosh (PES2UG24CS455)

Shaurya Ishan (PES2UG24CS464)

**PES University**

**Date: 5-09-2026**

**Table of Contents**

**Table of Contents      ii**

**Revision History      ii**

|           |                                           |          |
|-----------|-------------------------------------------|----------|
| <b>1.</b> | <b>Introduction</b>                       | <b>1</b> |
| 1.1       | Purpose                                   | 1        |
| 1.2       | Intended Audience and Reading Suggestions | 1        |
| 1.3       | Product Scope                             | 1        |
| 1.4       | References                                | 2        |
| <b>2.</b> | <b>Overall Description</b>                | <b>2</b> |
| 2.1       | Product Perspective                       | 2        |
| 2.2       | Product Functions                         | 2        |
| 2.3       | User Classes and Characteristics          | 2        |
| 2.4       | Operating Environment                     | 3        |
| 2.5       | Design and Implementation Constraints     | 3        |
| 2.6       | Assumptions and Dependencies              | 3        |
| <b>3.</b> | <b>External Interface Requirements</b>    | <b>4</b> |
| 3.1       | User Interfaces                           | 4        |
| 3.2       | Software Interfaces                       | 4        |
| 3.3       | Communications Interfaces                 | 5        |
| <b>4.</b> | <b>Analysis Models</b>                    | <b>5</b> |
| <b>5.</b> | <b>System Features</b>                    | <b>5</b> |
| 5.1       | Authentication and Role Management        | 5        |
| 5.2       | Event Creation and Management             | 6        |
| 5.3       | Event Discovery and Search                | 7        |
| 5.4       | Event Registration                        | 7        |
| 5.5       | Registration History                      | 8        |
| 5.6       | Participation Management                  | 8        |
| 5.7       | Attendance                                | 9        |
| 5.8       | Event Status and Capacity                 | 9        |
| 5.9       | Reporting                                 | 10       |

## **6. Other Nonfunctional Requirements 10**

6.1 Performance Requirements 10

6.2 Safety Requirements 10

6.3 Security Requirements 11

6.4 Software Quality Attributes 11

6.5 Business Rules 11

## **7. Other Requirements 12**

Appendix A: Glossary 12

Appendix B: Field Layouts 13

Appendix C: Requirement Traceability Matrix 14

## **Revision History**

| <b>Name</b> | <b>Date</b> | <b>Reason For Changes</b> | <b>Version</b> |
|-------------|-------------|---------------------------|----------------|
| Team-7      | 4-09-2026   | Initial SRS               | 1.0            |

## **Introduction**

### **Purpose**

This document specifies the software requirements for the PES University Event Management System (PESU-EMS), Release 1.0. PESU-EMS is a web-based platform for creating and publishing campus events at PES University, and for handling student registration and attendance for those events.

This SRS covers the complete Release 1.0 system: the student-facing web client, the organizer console, the backend REST API, and the database. It defines the functional behaviour, external interfaces, and non-functional constraints for all user roles of the system.

The following capabilities are explicitly outside the scope of this document and are deferred to later releases: online payment collection for paid events, native mobile applications, live integration with PESU Academy or the university ERP, and automated certificate generation.

## Intended Audience

This SRS is written for the following classes of reader:

Developers, who will use it as the authoritative build contract for the client, server and database layers.

Project reviewers and course faculty, who will assess the completeness, consistency and feasibility of the stated requirements.

Test engineers, who will derive test cases from the functional and non-functional requirements, each of which is stated so as to be independently verifiable.

Event organizers, comprising student club office-bearers and department coordinators, who will read the introductory and overall description sections to confirm that the system reflects how campus events are actually run.

System administrators, who will read the operating environment, constraints and security sections to plan deployment and ongoing operation.

Section 1 states the purpose, audience, scope and references.

Section 2 gives the overall product context, covering product perspective, a summary of major functions, user classes, the operating environment, design and implementation constraints, and assumptions and dependencies.

Section 3 specifies the external interface requirements, including user interfaces, software interfaces and communications interfaces.

Section 4 presents the analysis models, identifying primary actors and representative use cases.

Section 5 details the system features and their associated functional requirements, each carrying a unique identifier.

Section 6 specifies the non-functional requirements, covering performance, safety, security and software quality attributes.

Section 7 states other requirements, including database requirements.

## Product Scope

The PES University Event Management System is a web-based platform that centralizes the creation, promotion, discovery, and registration of university events. Currently, event information and registrations are scattered across multiple communication channels (WhatsApp groups, Google

Forms, notice boards, email), making it difficult for students to discover and register for events and for organizers to manage registrations and attendance.

The system will allow event organizers to create and manage events and track participation, while allowing students to discover, search, and register for events, and view their own participation history. The goal is to improve event visibility, simplify registration, and give organizers reliable attendance and registration analytics.

## References

MongoDB, *Integrate MongoDB with React – MERN Stack Documentation*.

React, *React Documentation – Quick Start*.

Node.js, *Node.js Documentation*.

Express.js, *Express.js Documentation*.

Mongoose, *Mongoose Documentation*.

MongoDB, *MongoDB Documentation*

## Overall Description

### Product Perspective

The Event Management System is a new, self-contained web application; it is not a follow-on to an existing system. It will replace informal event-communication channels (WhatsApp/Discord groups, printed posters, and standalone Google Forms) currently used within the university. The system consists of a web front end, an application/business logic layer, and a backend database, and it authenticates users against college-issued credentials.

### Product Functions

At a high level, the system lets users:

- Sign in and be authenticated using college credentials, with distinct permissions for students and organizers.
- Create, edit, and cancel events.
- Browse, search, and discover past and upcoming events.
- Register for events, subject to capacity limits, and view registration history
- View registered participants and mark attendance
- View analytics/reports on registrations and attendance

### User Classes and Characteristics

| User Class | Characteristics                                                                                                                                                                                                                                                                                          |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Students   | <ul style="list-style-type: none"> <li>• Primary, high-frequency users.</li> <li>• Browse and search events, register, and track their own participation history.</li> <li>• Assumed to have basic web/mobile literacy and a valid college login</li> </ul>                                              |
| Organizers | <ul style="list-style-type: none"> <li>• Students or staff acting as event owners/club representatives.</li> <li>• Create and manage events, monitor registrations and capacity, track attendance, and view reports.</li> <li>• Moderate frequency of use, concentrated around event windows.</li> </ul> |

## Operating Environment

The system is a web-based application accessible through modern desktop and mobile browsers (Chrome, Firefox, Edge, Safari). It runs on a server-side runtime with a relational database backend and is deployed on standard university infrastructure. No native mobile application is in scope for Phase-1.

## Design and Implementation Constraints

The following constraints apply:

1. The system must be implemented as a web-based application.
2. The system must implement role-based authorization. Authentication must integrate with, or validate against, official college credential formats.
3. Students must not be able to access organizer-only functionality.
4. Organizers must only be able to modify events they are authorized to manage.
5. Passwords must not be stored as plain text.
6. Registration capacity must be enforced by the backend.
7. The system should follow a modular architecture so different team members can develop separate components.
8. The system must be implemented within the academic term timeline of the mini-project.

## Assumptions and Dependencies

### Assumptions:

1. Every user has a unique account with valid login credentials.
2. Organizers are authorized users of the system.
3. Users have access to a modern web browser.
4. The server and database are available when the application is being used.
5. Event organizers provide accurate event information.
6. Attendance will be entered manually by the organizer.
7. Event capacity is specified when creating an event.

8. Events have a defined registration deadline.

### Dependencies:

The system depends on:

- React.js
- Node.js
- Express.js
- MongoDB
- Browser support
- Database connectivity

## External Interface Requirements

### User Interfaces

The system provides three principal interface areas: a Login screen, a Student dashboard (event discovery/search, registration, registration history), and an Organizer dashboard (event creation/cancellation, participant list, attendance marking, reports).

### Login Page

Contains:

- Email/username field.
- Password field.
- Login button.
- Error message for invalid login

### Student Pages

1. Student Dashboard
2. Browse Events
3. Event Details
4. My Registrations

### Organizer Pages

1. Organizer Dashboard
2. My Events
3. Create Event
4. Edit/Manage Event
5. Participants and Attendance
6. Reports

### Software Interfaces

The React frontend communicates with the Node.js/Express backend using REST APIs. The React web client communicates with the Node.js/Express.js backend over HTTPS using a RESTful JSON API. The Express backend interfaces with a MongoDB database via Mongoose for persistent storage of users, events, and registrations.

## Communications Interfaces

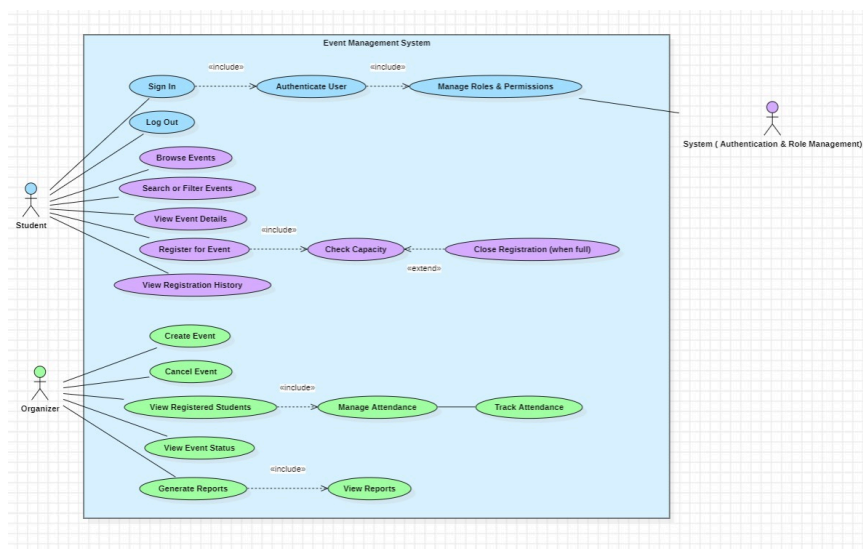
The system will use:

- HTTP/HTTPS for client-server communication.
- REST API conventions.
- JSON for data exchange.
- TCP/IP networking through standard web protocols.

HTTPS will be used in deployment to protect credentials and user information during transmission.

## Analysis Models

Primary actors are Student and Event Organizer. Representative use cases include: Sign In, Search/Browse Events, Register for Event, View Registration History, Create Event, Cancel Event, View Participants, Mark Attendance, and View Reports.



## System Features

### Authentication and Role Management

#### 5.1.1 Description and Priority

This feature authenticates users and provides functionality according to their assigned role.

Priority: High

#### 5.1.2 Stimulus/Response Sequences

## Successful Login

1. User enters email and password.
2. User selects Login.
3. System validates credentials.
4. System identifies the user's role.
5. System creates an authenticated session/token.
6. System redirects the user to the appropriate dashboard.

## Invalid Login

1. User enters invalid credentials.
2. System rejects the request.
3. System displays an error message.
4. User remains on the login page.

### 5.1.3 Functional Requirements

**AUTH-01:** The system shall allow registered users to sign in.

**AUTH-02:** The system shall validate supplied credentials.

**AUTH-03:** The system shall securely hash user passwords.

**AUTH-04:** The system shall identify the user's role.

**AUTH-05:** The system shall provide role-based access control.

**AUTH-06:** The system shall prevent students from accessing organizer-only functionality.

**AUTH-07:** The system shall reject invalid credentials.

## 5.2 Event Creation and Management

### 5.2.1 Description and Priority

Allows organizers to create, modify and cancel events.

Priority: High

### 5.2.2 Stimulus/Response Sequences

#### Event Creation

1. Organizer selects Create Event.
2. Organizer enters event information.
3. System validates the information.
4. System stores the event in MongoDB.
5. System displays the created event.

## Event Cancellation

1. Organizer selects an event.
2. Organizer selects Cancel Event.
3. System requests confirmation.
4. Organizer confirms cancellation.
5. System changes the event status to Cancelled.

### 5.2.3 Functional Requirements

**EVENT-01:** The system shall allow organizers to create events.

**EVENT-02:** The system shall require mandatory event information.

**EVENT-03:** The system shall validate event information.

**EVENT-04:** The system shall allow organizers to edit events they manage.

**EVENT-05:** The system shall allow organizers to cancel events they manage.

**EVENT-06:** The system shall maintain an event status.

**EVENT-07:** The system shall prevent unauthorized event modification.

**EVENT-08:** The system shall reject invalid event dates and capacity values.

## 5.3 Event Discovery and Search

### 5.3.1 Description and Priority

Allows users, particularly students, to discover university events.

Priority: High

### 5.3.2 Stimulus/Response Sequences

1. User opens Browse Events.
2. System retrieves events.
3. User enters a search query or selects filters.
4. System processes the query.
5. System displays matching events.

### 5.3.3 Functional Requirements

**DISC-01:** The system shall display available events.

**DISC-02:** The system shall allow users to search events.

**DISC-03:** The system shall allow users to filter events by category.

**DISC-04:** The system shall allow users to view event details.

**DISC-05:** The system shall display event status.

**DISC-06:** The system shall display registration availability.

## 5.4 Event Registration

### 5.4.1 Description and Priority

Allows students to register for available events.

Priority: High

### 5.4.2 Stimulus/Response Sequences

1. Student opens an event.
2. Student selects Register.
3. System verifies authentication.
4. System checks event status.
5. System checks registration deadline.
6. System checks event capacity.
7. System checks for an existing registration.
8. System creates the registration.
9. System displays registration success.

### 5.4.3 Functional Requirements

**REG-01:** The system shall allow authenticated students to register for events.

**REG-02:** The system shall prevent unauthenticated registration.

**REG-03:** The system shall prevent duplicate registrations.

**REG-04:** The system shall prevent registration after the deadline.

**REG-05:** The system shall prevent registration when capacity is reached.

**REG-06:** The system shall store registration information in MongoDB.

**REG-07:** The system shall allow students to cancel their registration.

**REG-08:** The system shall update event availability after registration changes.

## 5.5 Registration History

### 5.5.1 Description and Priority

Allows students to view their past and current registrations.

Priority: Medium

### **5.5.2 Stimulus/Response Sequence**

1. Student opens My Registrations.
2. System retrieves registrations belonging to the student.
3. System displays registration information.

### **5.5.3 Functional Requirements**

**HIST-01:** The system shall display a student's event registrations.

**HIST-02:** The system shall display registration status.

**HIST-03:** The system shall display event details associated with each registration.

**HIST-04:** The system shall distinguish active and cancelled registrations.

## **5.6 Participation Management**

### **5.6.1 Description and Priority**

Allows organizers to view students registered for their events.

Priority: High

### **5.6.2 Stimulus/Response Sequence**

1. Organizer selects an event.
2. Organizer opens Participants.
3. System retrieves registrations.
4. System displays registered students.

### **5.6.3 Functional Requirements**

**PART-01:** The system shall allow organizers to view registered participants.

**PART-02:** The system shall display relevant participant information.

**PART-03:** The system shall only allow authorized organizers to view participant lists.

**PART-04:** The system shall associate each participant with the correct event.

## **5.7 Attendance**

### 5.7.1 Description and Priority

Allows organizers to manually record attendance.

**Priority:** High

### 5.7.2 Stimulus/Response Sequence

1. Organizer opens the participant list.
2. System displays registered students.
3. Organizer marks each student as Present or Absent.
4. System stores attendance.
5. Organizer may modify the record if required.

### 5.7.3 Functional Requirements

**ATT-01:** The system shall allow organizers to mark registered students as Present.

**ATT-02:** The system shall allow organizers to mark registered students as Absent.

**ATT-03:** The system shall store attendance information.

**ATT-04:** The system shall associate attendance with the corresponding registration.

**ATT-05:** The system shall allow authorized organizers to modify attendance.

**ATT-06:** The system shall prevent unauthorized users from modifying attendance.

## 5.8 Event Status and Capacity

### 5.8.1 Description and Priority

Maintains event registration status and capacity.

**Priority:** High

### 5.8.2 Stimulus/Response Sequence

1. Student attempts to register.
2. System retrieves event capacity.
3. System checks current registrations.
4. If capacity is reached, registration is rejected.
5. System displays the event as Full.

### 5.8.3 Functional Requirements

**STATUS-01:** The system shall maintain an event status.

**STATUS-02:** The system shall identify when an event reaches capacity.

**STATUS-03:** The system shall prevent registrations when an event is full.

**STATUS-04:** The system shall prevent registration for cancelled events.

**STATUS-05:** The system shall display event status to users.

## 5.9 Reporting

### 5.9.1 Description and Priority

Provides organizers with statistics related to event registration and attendance.

Priority: Medium

### 5.9.2 Stimulus/Response Sequence

1. Organizer opens Reports.
2. System retrieves registration and attendance data.
3. System calculates statistics.
4. System displays the report.

### 5.9.3 Functional Requirements

**REP-01:** The system shall provide registration statistics.

**REP-02:** The system shall provide attendance statistics.

**REP-03:** The system shall display total registrations.

**REP-04:** The system shall display total attendance.

**REP-05:** The system shall display total absences.

## Other Nonfunctional Requirements

### Performance Requirements

- Normal API requests should respond within approximately 2 seconds under expected load.
- The system shall support multiple concurrent users.
- Registration operations shall maintain capacity consistency when multiple users attempt to register simultaneously.

### Safety Requirements

The system is not safety-critical so these requirements are not applicable to our project.

### Security Requirements

- The system shall authenticate users before accessing protected functionality.
- Passwords shall be stored using secure hashing.
- The system shall implement role-based authorization.
- HTTPS shall be used in production.

### Software Quality Attributes

## Usability

- The interface should be simple and intuitive.
- Event information should be clearly presented.
- Registration should require minimal steps.

## Reliability

- Registration data should remain consistent.
- Attendance records should accurately correspond to registrations.

## Maintainability

- Frontend and backend should be modular.
- API routes should follow consistent naming conventions.
- Components should have clearly defined responsibilities.

## Scalability

- The application should support increasing numbers of users and events.

## Portability

- The application should operate across modern web browsers.

## Business Rules

- Only users with an organizer role may create, edit, or cancel events.
- A student may not register for an event once its capacity has been reached (FR-9).
- A cancelled event shall not accept further registrations and shall be clearly marked as cancelled to students.
- Only the organizer who created an event (or an authorized administrator) may cancel it or mark attendance for it.

## Other Requirements

Database requirements: the system requires persistent storage for user accounts, events, and registration/attendance records in MongoDB. Because MongoDB is a NoSQL, document-oriented database rather than a relational one, referential integrity between students, events, and registrations is not enforced by the database itself and must instead be validated at the application layer.

We do not have any internationalization or legal or compliance requirements.

## Appendix A: Glossary

| Term         | Definition                                                                                                                |
|--------------|---------------------------------------------------------------------------------------------------------------------------|
| Organizer    | A user role authorized to create, edit, cancel events and manage their registrations and attendance.                      |
| Student      | A user role that can browse, search, and register for events.                                                             |
| Capacity     | The maximum number of participants an event can accept.                                                                   |
| Event Status | The current state of an event, e.g., Upcoming, Ongoing, Completed, or Cancelled.                                          |
| Registration | A record linking a student to an event they intend to attend.                                                             |
| Attendance   | A record indicating whether a registered student actually attended the event.                                             |
| MERN Stack   | MongoDB, Express.js, React, and Node.js.                                                                                  |
| Node.js      | The JavaScript runtime on which the backend server executes.                                                              |
| Express.js   | The Node.js web framework used to implement the backend REST API and route/middleware logic.                              |
| React        | The JavaScript library used to build the front-end single-page application.                                               |
| MongoDB      | The NoSQL, document-oriented database used to persist users, events, and registrations as JSON-like documents.            |
| Mongoose     | An Object Data Modeling library that defines schemas and provides validation for MongoDB documents accessed from Node.js. |
| REST API     | The architectural style used for the HTTP endpoints exposed by the Express backend and consumed by the React client.      |

## Appendix B: Field Layouts

Events:

| Field       | Length | Data Type | Description                                                         | Mandatory |
|-------------|--------|-----------|---------------------------------------------------------------------|-----------|
| _id         | 24     | ObjectId  | Unique identifier for the event document, auto-generated by MongoDB | Y         |
| Title       | 100    | String    | Name of the event                                                   | Y         |
| Description | 1000   | String    | Details about the event                                             | N         |
| Date/Time   | 8      | DateTime  | Scheduled date and time of the event                                | Y         |
| Venue       | 150    | String    | Location of the event                                               | Y         |
| Capacity    | 5      | Numeric   | Maximum number of participants                                      | Y         |
| Status      | 20     | String    | Upcoming / Ongoing / Completed / Cancelled                          | Y         |

|              |    |          |                                                                        |   |
|--------------|----|----------|------------------------------------------------------------------------|---|
| Organizer ID | 24 | ObjectId | Reference to the creating organizer's document in the users collection | Y |
|--------------|----|----------|------------------------------------------------------------------------|---|

Registration:

| Field             | Length | Data Type | Description                                                                | Mandatory |
|-------------------|--------|-----------|----------------------------------------------------------------------------|-----------|
| _id               | 24     | ObjectId  | Unique identifier for the registration document, auto-generated by MongoDB | Y         |
| Student ID        | 24     | ObjectId  | Reference to the registering student's document in the users collection    | Y         |
| Event ID          | 24     | ObjectId  | Reference to the corresponding document in the events collection           | Y         |
| Registration Date | 8      | Date      | Date the student registered                                                | Y         |
| Attendance Status | 15     | String    | Present / Absent / Not Marked                                              | N         |

## Appendix C: Requirement Traceability Matrix

| SI | Requirement ID | Brief Description of Requirement                                               | Architecture Reference                                | Design Reference                        | Test Case ID | System Test Case ID |
|----|----------------|--------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------|--------------|---------------------|
| 1  | AUTH-01        | The system shall allow registered users to sign in.                            | Frontend: Login/Auth<br>UI Backend: Auth Service      | UC-01 Sign In                           | TC-AUTH-01   | STC-AUTH-01         |
| 2  | AUTH-02        | The system shall validate supplied credentials.                                | Frontend: Login/Auth<br>UI Backend: Auth Service      | UC-01 Sign In                           | TC-AUTH-02   | STC-AUTH-02         |
| 3  | AUTH-03        | The system shall securely hash user passwords.                                 | Frontend: Login/Auth<br>UI Backend: Auth Service      | UC-01 Sign In                           | TC-AUTH-03   | STC-AUTH-03         |
| 4  | AUTH-04        | The system shall identify the user's role.                                     | Frontend: Login/Auth<br>UI Backend: Auth Service      | UC-01 Sign In                           | TC-AUTH-04   | STC-AUTH-04         |
| 5  | AUTH-05        | The system shall provide role-based access control.                            | Frontend: Login/Auth<br>UI Backend: Auth Service      | UC-01 Sign In                           | TC-AUTH-05   | STC-AUTH-05         |
| 6  | AUTH-06        | The system shall prevent students from accessing organizer-only functionality. | Frontend: Login/Auth<br>UI Backend: Auth Service      | UC-01 Sign In                           | TC-AUTH-06   | STC-AUTH-06         |
| 7  | AUTH-07        | The system shall reject invalid credentials.                                   | Frontend: Login/Auth<br>UI Backend: Auth Service      | UC-01 Sign In                           | TC-AUTH-07   | STC-AUTH-07         |
| 8  | EVENT-01       | The system shall allow organizers to create events.                            | Frontend: Organizer Console<br>Backend: Event Service | UC-05 Create Event / UC-06 Cancel Event | TC-EVENT-01  | STC-EVENT-01        |

|    |          |                                                                       |                                                          |                                         |             |              |
|----|----------|-----------------------------------------------------------------------|----------------------------------------------------------|-----------------------------------------|-------------|--------------|
| 9  | EVENT-02 | The system shall require mandatory event information.                 | Frontend: Organizer Console Backend: Event Service       | UC-05 Create Event / UC-06 Cancel Event | TC-EVENT-02 | STC-EVENT-02 |
| 10 | EVENT-03 | The system shall validate event information.                          | Frontend: Organizer Console Backend: Event Service       | UC-05 Create Event / UC-06 Cancel Event | TC-EVENT-03 | STC-EVENT-03 |
| 11 | EVENT-04 | The system shall allow organizers to edit events they manage.         | Frontend: Organizer Console Backend: Event Service       | UC-05 Create Event / UC-06 Cancel Event | TC-EVENT-04 | STC-EVENT-04 |
| 12 | EVENT-05 | The system shall allow organizers to cancel events they manage.       | Frontend: Organizer Console Backend: Event Service       | UC-05 Create Event / UC-06 Cancel Event | TC-EVENT-05 | STC-EVENT-05 |
| 13 | EVENT-06 | The system shall maintain an event status.                            | Frontend: Organizer Console Backend: Event Service       | UC-05 Create Event / UC-06 Cancel Event | TC-EVENT-06 | STC-EVENT-06 |
| 14 | EVENT-07 | The system shall prevent unauthorized event modification.             | Frontend: Organizer Console Backend: Event Service       | UC-05 Create Event / UC-06 Cancel Event | TC-EVENT-07 | STC-EVENT-07 |
| 15 | EVENT-08 | The system shall reject invalid event dates and capacity values.      | Frontend: Organizer Console Backend: Event Service       | UC-05 Create Event / UC-06 Cancel Event | TC-EVENT-08 | STC-EVENT-08 |
| 16 | DISC-01  | The system shall display available events.                            | Frontend: Discovery UI Backend: Event Service            | UC-02 Search/Browse Events              | TC-DISC-01  | STC-DISC-01  |
| 17 | DISC-02  | The system shall allow users to search events.                        | Frontend: Discovery UI Backend: Event Service            | UC-02 Search/Browse Events              | TC-DISC-02  | STC-DISC-02  |
| 18 | DISC-03  | The system shall allow users to filter events by category.            | Frontend: Discovery UI Backend: Event Service            | UC-02 Search/Browse Events              | TC-DISC-03  | STC-DISC-03  |
| 19 | DISC-04  | The system shall allow users to view event details.                   | Frontend: Discovery UI Backend: Event Service            | UC-02 Search/Browse Events              | TC-DISC-04  | STC-DISC-04  |
| 20 | DISC-05  | The system shall display event status.                                | Frontend: Discovery UI Backend: Event Service            | UC-02 Search/Browse Events              | TC-DISC-05  | STC-DISC-05  |
| 21 | DISC-06  | The system shall display registration availability.                   | Frontend: Discovery UI Backend: Event Service            | UC-02 Search/Browse Events              | TC-DISC-06  | STC-DISC-06  |
| 22 | REG-01   | The system shall allow authenticated students to register for events. | Frontend: Event Details UI Backend: Registration Service | UC-03 Register for Event                | TC-REG-01   | STC-REG-01   |
| 23 | REG-02   | The system shall prevent unauthenticated registration.                | Frontend: Event Details UI Backend: Registration Service | UC-03 Register for Event                | TC-REG-02   | STC-REG-02   |
| 24 | REG-03   | The system shall prevent duplicate registrations.                     | Frontend: Event Details UI Backend: Registration Service | UC-03 Register for Event                | TC-REG-03   | STC-REG-03   |
| 25 | REG-04   | The system shall prevent registration after the deadline.             | Frontend: Event Details UI Backend: Registration Service | UC-03 Register for Event                | TC-REG-04   | STC-REG-04   |
| 26 | REG-05   | The system shall prevent registration when capacity is reached.       | Frontend: Event Details UI Backend: Registration Service | UC-03 Register for Event                | TC-REG-05   | STC-REG-05   |
| 27 | REG-06   | The system shall store registration information in MongoDB.           | Frontend: Event Details UI Backend: Registration Service | UC-03 Register for Event                | TC-REG-06   | STC-REG-06   |

|    |           |                                                                              |                                                             |                                               |              |               |
|----|-----------|------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------|--------------|---------------|
| 28 | REG-07    | The system shall allow students to cancel their registration.                | Frontend: Event Details UI Backend: Registration Service    | UC-03 Register for Event                      | TC-REG-07    | STC-REG-07    |
| 29 | REG-08    | The system shall update event availability after registration changes.       | Frontend: Event Details UI Backend: Registration Service    | UC-03 Register for Event                      | TC-REG-08    | STC-REG-08    |
| 30 | HIST-01   | The system shall display a student's event registrations.                    | Frontend: My Registrations UI Backend: Registration Service | UC-04 View Registration History               | TC-HIST-01   | STC-HIST-01   |
| 31 | HIST-02   | The system shall display registration status.                                | Frontend: My Registrations UI Backend: Registration Service | UC-04 View Registration History               | TC-HIST-02   | STC-HIST-02   |
| 32 | HIST-03   | The system shall display event details associated with each registration.    | Frontend: My Registrations UI Backend: Registration Service | UC-04 View Registration History               | TC-HIST-03   | STC-HIST-03   |
| 33 | HIST-04   | The system shall distinguish active and cancelled registrations.             | Frontend: My Registrations UI Backend: Registration Service | UC-04 View Registration History               | TC-HIST-04   | STC-HIST-04   |
| 34 | PART-01   | The system shall allow organizers to view registered participants.           | Frontend: Participants UI Backend: Registration Service     | UC-07 View Participants                       | TC-PART-01   | STC-PART-01   |
| 35 | PART-02   | The system shall display relevant participant information.                   | Frontend: Participants UI Backend: Registration Service     | UC-07 View Participants                       | TC-PART-02   | STC-PART-02   |
| 36 | PART-03   | The system shall only allow authorized organizers to view participant lists. | Frontend: Participants UI Backend: Registration Service     | UC-07 View Participants                       | TC-PART-03   | STC-PART-03   |
| 37 | PART-04   | The system shall associate each participant with the correct event.          | Frontend: Participants UI Backend: Registration Service     | UC-07 View Participants                       | TC-PART-04   | STC-PART-04   |
| 38 | ATT-01    | The system shall allow organizers to mark registered students as Present.    | Frontend: Attendance UI Backend: Attendance Service         | UC-08 Mark Attendance                         | TC-ATT-01    | STC-ATT-01    |
| 39 | ATT-02    | The system shall allow organizers to mark registered students as Absent.     | Frontend: Attendance UI Backend: Attendance Service         | UC-08 Mark Attendance                         | TC-ATT-02    | STC-ATT-02    |
| 40 | ATT-03    | The system shall store attendance information.                               | Frontend: Attendance UI Backend: Attendance Service         | UC-08 Mark Attendance                         | TC-ATT-03    | STC-ATT-03    |
| 41 | ATT-04    | The system shall associate attendance with the corresponding registration.   | Frontend: Attendance UI Backend: Attendance Service         | UC-08 Mark Attendance                         | TC-ATT-04    | STC-ATT-04    |
| 42 | ATT-05    | The system shall allow authorized organizers to modify attendance.           | Frontend: Attendance UI Backend: Attendance Service         | UC-08 Mark Attendance                         | TC-ATT-05    | STC-ATT-05    |
| 43 | ATT-06    | The system shall prevent unauthorized users from modifying attendance.       | Frontend: Attendance UI Backend: Attendance Service         | UC-08 Mark Attendance                         | TC-ATT-06    | STC-ATT-06    |
| 44 | STATUS-01 | The system shall maintain an event status.                                   | Backend: Event Service + Registration Service               | UC-03 Register for Event / UC-05 Create Event | TC-STATUS-01 | STC-STATUS-01 |

|    |           |                                                               |                                                    |                                               |              |               |
|----|-----------|---------------------------------------------------------------|----------------------------------------------------|-----------------------------------------------|--------------|---------------|
| 45 | STATUS-02 | The system shall identify when an event reaches capacity.     | Backend: Event Service + Registration Service      | UC-03 Register for Event / UC-05 Create Event | TC-STATUS-02 | STC-STATUS-02 |
| 46 | STATUS-03 | The system shall prevent registrations when an event is full. | Backend: Event Service + Registration Service      | UC-03 Register for Event / UC-05 Create Event | TC-STATUS-03 | STC-STATUS-03 |
| 47 | STATUS-04 | The system shall prevent registration for cancelled events.   | Backend: Event Service + Registration Service      | UC-03 Register for Event / UC-05 Create Event | TC-STATUS-04 | STC-STATUS-04 |
| 48 | STATUS-05 | The system shall display event status to users.               | Backend: Event Service + Registration Service      | UC-03 Register for Event / UC-05 Create Event | TC-STATUS-05 | STC-STATUS-05 |
| 49 | REP-01    | The system shall provide registration statistics.             | Frontend: Reports UI<br>Backend: Reporting Service | UC-09 View Reports                            | TC-REP-01    | STC-REP-01    |
| 50 | REP-02    | The system shall provide attendance statistics.               | Frontend: Reports UI<br>Backend: Reporting Service | UC-09 View Reports                            | TC-REP-02    | STC-REP-02    |
| 51 | REP-03    | The system shall display total registrations.                 | Frontend: Reports UI<br>Backend: Reporting Service | UC-09 View Reports                            | TC-REP-03    | STC-REP-03    |
| 52 | REP-04    | The system shall display total attendance.                    | Frontend: Reports UI<br>Backend: Reporting Service | UC-09 View Reports                            | TC-REP-04    | STC-REP-04    |
| 53 | REP-05    | The system shall display total absences.                      | Frontend: Reports UI<br>Backend: Reporting Service | UC-09 View Reports                            | TC-REP-05    | STC-REP-05    |